

Autonomous Software Agents for Deliveroo.js

BDI agent, LLM agent, coordination and PDDL planning

Matteo Gervasi
matteogervasi2000@gmail.com
Student ID: TODO

June 16, 2026

Repository: <https://github.com/Giambac/asa-deliveroo-agent>

1 Introduction

Deliveroo.js is a grid-world game in which autonomous agents pick up parcels and deliver them to delivery tiles for points. Parcel rewards decay over time, actions are serialized by the server, and the world is only partially observable through local sensing. The exam task is to build **Agent A**, a BDI agent for *Challenge 1* (single-agent parcel farming on eight maps), and **Agent B**, an LLM-driven agent for *Challenge 2* (agents that shout natural-language missions), together with Agent A – Agent B coordination and a meaningful **PDDL** planning component.

We built a single ES-module codebase shared by both agents: belief revision, option generation, swappable utility-based strategies, intention revision with hysteresis, and a plan library with deterministic BFS movement on the directed map graph, plus an optional PDDL plan for the `go_to` sub-goal. Agent B adds, on top of the same machinery, an LLM mission interpreter and a structured team protocol.

The headline Challenge 1 result is that a deliberately simple **greedy-nearest** baseline is the strongest of four strategies across all eight maps: these maps reward raw delivery throughput, and the value-aware strategies lose because they deliver in small batches. We reached this conclusion through a systematic benchmark and a log-level diagnosis that also falsified our initial “slow decay favours batching” hypothesis — a worked example of using the metrics pipeline for analysis, not only scoring. The rest of the report describes the architecture (Section 2), Agent A and its strategies (Section 3), the LLM agent, PDDL and coordination (Sections 4–6), and the experiments (Section 7).

2 Architecture

The two agents share one BDI pipeline; only the top of the stack differs (Agent B adds the LLM and team layers). Each server event triggers the loop:

```
socket events (map, config, you, sensing, msg)
|
+-- BeliefBase      map graph, me, parcels, agents, mission, claims
|
+-- OptionGenerator what is possible (pickup / deliver / explore / wait)
|
+-- Strategy        what is preferable (swappable; LLM adapts it via
|                   mission state for Agent B)
|
+-- IntentionRevision commit / revise (replace policy + hysteresis)
|
+-- PlanLibrary     how: GoPickUp, DeliverCarried, Explore,
|                   PddlGoTo -> FollowPathGoTo (BFS fallback)
|
+-- ActionExecutor  serialized move/pickup/putdown, red-light gate
```

Control loop. On each `sensing`/you event (and a periodic timer, since sensing is change-driven) the agent revises beliefs, regenerates options, and lets intention revision decide whether to keep the current intention or switch. A new option must beat the current one by a hysteresis margin before the agent commits, which prevents target thrashing. The chosen intention is achieved by the first applicable plan in the plan library; failures fall through to the next plan.

Design principles. (i) *Infrastructure vs. strategy*: strategies only score options; they never move the agent, so a new strategy is one file (Section 3). (ii) *Deterministic low-level movement*: pathfinding is BFS on a directed graph; the LLM stays high-level and never drives motion. (iii) *Serialized actions*: all actuator calls are chained into a promise queue to respect the server’s action mutex.

Repository layout.

Path	Role
<code>src/core/</code>	BeliefBase, OptionGenerator, IntentionRevision, Intention, PlanLibrary, ActionExecutor
<code>src/planning/</code>	GridGraph (directed, arrow tiles), PathPlanner (BFS), PddlPlanner
<code>src/strategies/</code>	four baseline strategies + a registry (Section 3)
<code>src/llm/</code>	mission interpreter + tools (Agent B, Section 4)
<code>src/communication/</code>	team protocol (Section 6)
<code>src/metrics/</code>	run logging and result aggregation
<code>scripts/</code>	run / experiment / baseline / campaign / smoke-test
<code>experiments/</code>	results and the committed baseline write-up

Verification tooling. An offline smoke test (`npm test`, no server or network) pins the load-bearing semantics: directed-graph construction with one-way arrow tiles, BFS and reversed multi-source delivery distances, belief revision (whole-tick decay projection, negative evidence), strategy selection, mission parsing, PDDL problem generation, and ack normalization. It runs in milliseconds and gates every change.

Robustness note. We never trust a single server ack shape: the running course servers differ from the SDK type declarations. Pickup/putdown acks may carry `{id}` objects, plain strings, or no usable id at all (observed both locally and on the course server). All ack ids pass through a normalization layer, with belief-reconciliation fallbacks when the server contradicts the carry belief — a fix prompted by a live “phantom-carry” loop (Section 3).

3 Agent A: the BDI agent

Beliefs. The map is treated as static known knowledge: the graph is built once from the `map` event, with walls removed, delivery and spawner tiles tagged, and one-way arrow tiles encoded as directed edges (an edge is omitted when it opposes a tile’s arrow). Parcels and other agents come from *partial* sensing and are stamped with a `lastSeen` time. A parcel’s reward is projected forward by subtracting whole decay ticks elapsed since it was last seen (matching the server’s whole-tick semantics, not a fractional estimate). Negative evidence is applied: a remembered parcel that is absent from a now-visible tile is deleted.

Belief reconciliation — a live finding. Early live testing exposed a concrete “my knowledge was wrong, not the world” case. The running server’s pickup acks carried no `id` field (unlike the SDK types), so the agent marked a phantom parcel as carried and then issued 336 futile `putdown` retries in one 60s run, scoring 173. The fix normalizes all ack ids through one layer and reconciles beliefs against the server (`markTilePickedUp/clearCarried`) when no id is usable; the same scenario then scored 800 with 30 deliveries and zero futile retries. The empty ack shape was later confirmed on the course server too.

Options and strategies. The option generator emits `go_pick_up`, `deliver_carried`, `explore` and `wait` (plus `go_to_mission_target` for Agent B). A strategy assigns each option a utility and `selectOption` takes the argmax, so strategies decide but never act. Four baselines are implemented: **greedy-nearest**

(nearest parcel; deliver only when nothing is reachable), **reward-distance** (maximize projected *delivered* value, reward minus decay $\times$ distance), **delivery-threshold** (batch until N parcels, then deliver), and **mission-aware** (**reward-distance** plus mission constraints, Agent B’s default). A post-hoc **reward-distance-total** variant is discussed in Section 7. Adding a strategy is a single file plus a registry entry; the rest of the pipeline is untouched.

Intention revision. Revision follows a replace policy with an additive hysteresis margin: a candidate option must exceed the current intention’s utility by the margin before the agent switches, damping target thrashing when several parcels have similar value. An intention stops when achieved, impossible, or no longer worthwhile; on commit to a pickup the agent announces a claim to its teammate (Section 6).

Plans and failure handling. An intention is achieved by the first applicable plan in the library, with the rest as fallback (e.g. the PDDL `go_to` plan falls back to BFS path following). Movement tolerates a few failed moves (dynamic obstacles) before declaring a tile soft-blocked so the next BFS routes around it. Shutdown is close-safe: tearing the agent down mid-plan (e.g. between benchmark runs) cannot crash the process.

Partial observability and limits. The agent therefore has full knowledge of the static layout but only partial, decaying knowledge of parcels and opponents; when no parcel is worth pursuing it explores the least-recently-seen spawner. It does *not* model crates or pushable obstacles, so maps with crate mechanics are out of scope — confirmed by a course-server check where the live `crates_one_way` map left most tiles unreachable (Section 7).

4 Agent B: the LLM agent

5 PDDL planning

Where PDDL sits. PDDL is integrated as a *member of the plan library* (Section 2), not as a replacement for the reactive loop. The `go_to` intention can be achieved either by the fast BFS plan (FollowPathGoTo) or by a PDDL plan (PddlGoTo) generated from current beliefs; both yield the same kind of deterministic move sequence, executed by the same `ActionExecutor`. This keeps PDDL meaningful — it genuinely plans the *means* of an intention — while task selection (which parcel, when to deliver) stays with the BDI strategy. The same domain also expresses a full single-parcel collect-and-deliver subproblem (PddlPickUpAndDeliver), used for experiments and gated off by default.

Domain. The STRIPS domain models a single agent moving on the directed tile graph and, optionally, picking up and delivering one parcel. One key encoding choice: movement is a set of *directed* edge predicates (`up/down/left/right`), emitted only where entry is allowed, so one-way arrow tiles are captured for free — a forbidden direction simply has no edge, and no special action is needed. Pickup and putdown extend the same domain to a delivery task:

```
(:predicates (at ?t) (parcel ?p) (parcel-at ?p ?t)
              (carrying ?p) (delivery ?t) (delivered ?p)
              (up ?from ?to) (down ?from ?to) (left ?from ?to) (right ?from ?to))

(:action move-up      :parameters (?from ?to)
  :precondition (and (at ?from) (up ?from ?to))
  :effect (and (not (at ?from)) (at ?to))) ; one per direction

(:action pickup      :parameters (?p ?t)
  :precondition (and (parcel ?p) (at ?t) (parcel-at ?p ?t))
  :effect (and (not (parcel-at ?p ?t)) (carrying ?p)))

(:action putdown     :parameters (?p ?t)
  :precondition (and (parcel ?p) (at ?t) (delivery ?t) (carrying ?p))
  :effect (and (not (carrying ?p)) (delivered ?p)))
```

Problem generation from beliefs. A problem is built on demand from the BeliefBase. A BFS from the agent’s tile collects the reachable region; each tile becomes an object `t_x_y`, each allowed step an edge predicate. For a path problem the goal is (`at target`); for a delivery problem we add the parcel object, (`parcel-at p t`), every reachable (`delivery t`) tile, and the goal (`delivered p`) — letting the solver sequence pickup, routing and delivery in one symbolic plan. The region is capped at `PDDL_MAX_TILES` (1600) and the target/parcel must lie inside it, otherwise generation fails fast and BFS takes over.

When PDDL fits. PDDL is appropriate for *deterministic, fully-believed* subproblems and a poor fit for the outer loop, for two measured reasons. First, the world changes per frame (parcels decay, agents move), so a long symbolic plan goes stale within seconds. Second, the online solver (`@unitn-asa/pddl-client`, planning-as-a-service) adds a fixed latency of roughly 3 s per call — even for a one-step plan. We therefore call the solver only off the critical path, behind three guards: calls are time-limited (`PDDL_TIMEOUT_MS`, default 2.5 s, then fall back to BFS); short routes skip PDDL entirely (`PDDL_MIN_PATH_LENGTH`, default 10, since BFS is essentially instant there); and once parcels are being carried, PDDL is skipped because reward decay makes the solver delay especially expensive during delivery. Any failure (timeout, unreachable target, missing package) raises and the next plan in the library — BFS — serves the same intention, so PDDL is strictly optional at runtime.

Measured comparison. Table 1 reports illustrative 60 s live runs on the same scenario, contrasting BFS with PDDL movement and the full PDDL delivery task. A naive use of PDDL — calling the solver for every `go_to` — collapsed the score to 0: the plans were correct but each cost ~ 3 s, so the agent spent the run waiting instead of acting (16 planner calls, 0 deliveries). The selective gate above changed the movement result from 0 to 137 while cutting planner calls from 16 to 4. The full pickup-and-deliver task plan works (parcels are collected and delivered by a single solved plan) but remains solver-bound, so BFS/BDI stays the pragmatic default and PDDL is kept as a meaningful, bounded component and as report evidence.

Setup	Score	Delivered	Picked	Planner calls
BFS / BDI (default)	231	21	21	0
Naive PDDL movement	0	0	12	16
Selective PDDL movement	137	13	14	4
Full PDDL delivery task	36	5	8	14

Table 1: BFS vs. PDDL on one 60 s scenario (single representative runs). Planner calls and failures come from the `plannerCalls` metric and the `pddl_plan/pddl_failure` logs. See Section 7 for the broader strategy comparison.

Problem complexity. The generated problems are deliberately small and tractable: objects are bounded by the reachable region ($\leq \text{PDDL_MAX_TILES}$), edges are $O(\text{tiles})$ since each tile has at most four neighbours, and the path subproblem is shortest-path-equivalent (the same answer BFS computes in $O(V+E)$). The cost is not search difficulty but the network round-trip to the online solver; the delivery problem only adds one parcel and the delivery-tile set, so it stays within the same complexity class while letting the planner sequence pickup, routing and putdown together. Mission constraints (deliver-exactly- N , value thresholds) are intentionally *not* pushed into PDDL: they are enforced by the BDI plans (Section 3), which keeps the symbolic model simple and avoids regenerating problems as missions change.

Scope and future work. Crate-pushing and dynamic-obstacle variants (the Sokoban-style maps) are out of scope: the agent assumes a static known map with partial sensing of parcels and opponents (Section 3). A natural extension is a richer multi-parcel delivery domain with explicit capacity and mission constraints.

6 Coordination

7 Experiments and validation

Methodology. Because parcel spawns are random, every (scenario, strategy) pair is run five times and compared by mean score. The map is selected server-side per run; a campaign runner starts and stops the server for each map, runs every strategy with a fresh in-game identity (so scores start at 0 and never bleed across runs), and an aggregator produces the comparison tables. The Challenge 1 campaign covered all eight 26c1_* maps, four strategies, five 120s runs each (160 runs).

Baseline result. `greedy-nearest` is the strongest strategy: it wins outright on six maps and ties on the other two; it is never beaten.

Map	greedy	best other	outcome
26c1_1	868	742 (d.thr.)	greedy
26c1_2	866	691 (d.thr.)	greedy
26c1_3	377	397 (d.thr.)	tie (within noise)
26c1_4	828	657 (d.thr.)	greedy
26c1_5	319	337 (d.thr.)	tie (within noise)
26c1_6	917	622 (d.thr.)	greedy
26c1_7	882	290 (d.thr.)	greedy
26c1_8	1035	604 (d.thr.)	greedy

Diagnosis. A log-level analysis explains *why*. On the large maps `greedy` delivers about 7.6 parcels per drop-off because it delivers only when no parcel is reachable, hoarding on these parcel-rich maps, whereas the value-aware strategies deliver in ~ 2 -parcel batches and so collect far fewer parcels (on 26c1_7, 33.8 vs. 10.6 picked up). Crucially there are *no real plan failures* for either — the logged failures are all intention-revision cancellations — so this is a strategy/utility effect, not a pathfinding bug. This **falsified** our prior hypothesis that the slow-decay, high-reward maps (26c1_7/8) would favour batching: there `delivery-threshold` does *worst*, because on big maps the bottleneck is travel time, not decay.

A corrected false positive. The table suggested `mission-aware` and `reward-distance` differed on two maps, but inspection of the source shows their utility reduces to the same function when no mission is active, and the logs confirm identical behaviour (zero mission intentions). The apparent gap was small-sample noise — a reminder not to over-read $n=5$ significance.

A diagnosis-driven variant (preliminary). The diagnosis pinpointed a scale mismatch: `reward-distance` scores a pickup by the new parcel only, while its deliver utility already includes the carried value, biasing it toward early delivery. A post-hoc `reward-distance-total` variant scores a pickup by the whole load’s delivered value. In a preliminary smoke test ($n=3$) it roughly *doubles* the batch size (26c1_7: $2.7 \rightarrow 5.3$ parcels/delivery; 26c1_8: $1.3 \rightarrow 2.2$) and improves `reward-distance` (26c1_7: $249 \rightarrow 368$; 26c1_8: $409 \rightarrow 537$), confirming the diagnosis — but it still does not beat `greedy`. The original strategies and defaults were left unchanged; this is an experimental variant, not a redesign.

When greedy works, and where it would not. `greedy` wins because Challenge 1 rewards throughput (near-uniform rewards, frequent spawns). Its expected weaknesses — heterogeneous rewards, aggressive decay, tight capacity, contested parcels, one-way layouts, or external missions — are largely absent from this dataset; the only non-dominant maps are 26c1_3 and 26c1_5 (both ties), and the natural arena for a non-greedy strategy is Challenge 2, where compliance with missions, not farming, decides the score.

Live compatibility check. A run against the course server (`deliveroojs.bears.disi.unitn.it`) confirmed connection, token issuance, map loading and logging all work, and that pickup acks carry no ids there either (validating the reconciliation layer). The live map was `crates_one_way` (9×9); the agent found most tiles unreachable and scored 0 — a compatibility limit (no crate modeling), not a strategy benchmark.

8 Conclusion